Malware Analysis

Monnappa K A, Learning Malware Analysis, Packt Publishing Ltd, 2018..

Shubha Malige

Asst.Professor
CSE(Cyber Security)

Unit 1

Introduction to Malware Analysis: What is Malware? What is Malware Analysis? Why is Malware Analysis? Types of Malware Analysis. Static Analysis: Determining the File Type, Fingerprinting the Malware, Multiple Anti-Virus Scanning, Extracting Strings, Determining File Obfuscation.

- **Pedagogy: Chalk and board, Problem based learning, Demonstration.**

Introduction:

The number of cyber attacks is undoubtedly on the rise, targeting government, military, public and private sectors. These cyber attacks focus on targeting individuals or organizations with an effort to extract valuable information.

Sometimes, these cyber attacks are allegedly linked to cybercrime or state-sponsored groups, but may also be carried out by individual groups to achieve their goals. Most of these cyber attacks use malicious software (also called malware) to infect their targets.

What is Malware?

What is Malware?

- Malware is a code that performs malicious actions; it can take the form of an executable, script, code, or any other software.
- Attackers use malware to steal sensitive information, spy on the infected system, or take control of the system.
- It typically gets into your system without your consent and can be delivered via various communication channels such as email, web, or USB drives.

What is Malware?

The following are some of the malicious actions performed by malware:

- Disrupting computer operations
- Stealing sensitive information, including personal, business, and financial data
- Unauthorized access to the victim's system
- Spying on the victims
- Sending spam emails
- Engaging in distributed-denial-of-service attacks (DDOS)
- Locking up the files on the computer and holding them for ransom

What is Malware:

Malware is a broad term that refers to different types of malicious programs such as trojans, viruses, worms, and rootkits. some of these malicious programs are categorized based on their functionality and attack vectors as mentioned here:

Virus or Worm: Malware that is capable of copying itself and spreading to other computers. A virus needs user intervention, whereas a worm can spread without user intervention.

Trojan: Malware that disguises itself as a regular program to trick users to install it on their systems. Once installed, it can perform malicious actions such as stealing sensitive data, uploading files to the attacker's server, or monitoring webcams.

Backdoor / Remote Access Trojan (RAT): This is a type of Trojan that enables the attacker to gain access to and execute commands on the compromised system.

Adware: Malware that presents unwanted advertisements (ads) to the user. They usually get delivered via free downloads and can forcibly install software on your system.

What is Malware?

Botnet: This is a group of computers infected with the same malware (called bots), waiting to receive instructions from the command-and-control server controlled by the attacker. The attacker can then issue a command to these bots, which can perform malicious activities such as DDOS attacks or sending spam emails.

Information stealer: Malware designed to steal sensitive data such as banking credentials or typed keystrokes from the infected system. Some examples of these malicious programs include key loggers, spyware, sniffers, and form grabbers.

Ransomware: Malware that holds the system for ransom by locking users out of their computer or by encrypting their files.

Rootkit: Malware that provides the attacker with privileged access to the infected system and conceals its presence or the presence of other software.

Downloader or dropper: Malware designed to download or install additional malware components.

What is Malware Analysis?

Malware Analysis is the study of malware's behavior. The objective of malware analysis is to understand the working of malware and how to detect and eliminate it. It involves analyzing the suspect binary in a safe environment to identify its characteristics and functionalities so that better defenses can be built to protect an organization's network.

Malware analysis is the process of studying malicious software (malware) to understand what it is, how it works, what damage it can cause, and how to stop or detect it.

In Simple Words:
Malware analysis = breaking down a virus or malicious program to see its behavior and purpose.

Why is Malware Analysis?

The primary motive behind performing malware analysis is to extract information from the malware sample, which can help in responding to a malware incident. The goal of malware analysis is to determine the capability of malware, detect it, and contain it. It also helps in determining identifiable patterns that can be used to cure and prevent future infections.

Why is Malware Analysis?

The reasons why malware analysis is performed:

- To determine the nature and purpose of the malware. For example, it can help you determine whether malware is an information stealer, HTTP bot, spam bot, rootkit, keylogger, or RAT, and so on.

- To gain an understanding of how the system was compromised and its impact.

- To identify the network indicators associated with the malware, which can then be used to detect similar infections using network monitoring. For example, during your analysis, if you determine that a malware contacts a particular domain/IP address, then you can use this domain/IP address to create a signature and monitor the network traffic to identify all the hosts contacting that domain/IP address.

Why is Malware Analysis?

The reasons why malware analysis is performed:

- To extract host-based indicators such as filenames, and registry keys, which, in turn, can be used to determine similar infection using host-based monitoring. For instance, if you learn that a malware creates a registry key, you can use this registry key as an indicator to create a signature, or scan your network to identify the hosts that have the same registry key.

- To determine the attacker's intention and motive. For instance, during your analysis, if you find that the malware is stealing banking credentials, then you can deduce that the motive of the attacker is monetary gain.

Types of Malware Analysis?

Static Analysis:

This is the process of analyzing a binary without executing it. It is easiest to perform and allows you to extract the metadata associated with the suspect binary. Static analysis might not reveal all the required information, but it can sometimes provide interesting information that helps in determining where to focus your subsequent analysis efforts.

Dynamic analysis (Behavioral Analysis): This is the process of executing the suspect binary in an isolated environment and monitoring its behavior. This analysis technique is easy to perform and gives valuable insights into the activity of the binary during its execution. This analysis technique is useful but does not reveal all the functionalities of the hostile program.

Types of Malware Analysis?

Code analysis:

- It is an advanced technique that focuses on analyzing the code to understand the inner workings of the binary.
- This technique reveals information that is not possible to determine just from static and dynamic analysis.
- Code analysis is further divided into Static code analysis and Dynamic code analysis.
- Static code analysis involves disassembling the suspect binary and looking at the code to understand the program's behavior, whereas Dynamic code analysis involves debugging the suspect binary in a controlled manner to understand its functionality.
- Code analysis requires an understanding of the programming language and operating system concepts.

Types of Malware Analysis?

Memory analysis (Memory forensics):

- This is the technique of analyzing the computer's RAM for forensic artifacts. It is typically a forensic technique, but integrating it into your malware analysis will assist in gaining an understanding of the malware's behavior after infection.

- Memory analysis is especially useful to determine the stealth and evasive capabilities of the malware.

Static Analysis

- Static analysis is the technique of analyzing the suspect file without executing it.
- It is an initial analysis method that involves extracting useful information from the suspect binary to make an informed decision on how to classify or analyze it and where to focus your subsequent analysis efforts.

Static Analysis

1. Determining File Type:

- ✔ During your analysis, determining the file type of a suspect binary will help you identify the malware's target operating system (Windows, Linux, and so on) and architecture (32-bit or 64-bit platforms).

- ✔ For example, if the suspect binary has a file type of Portable Executable (PE), which is the file format for Windows executable files (.exe, .dll, .sys, .drv, .com,.ocx, and so on), then you can deduce that the file is designed to target the Windows operating system.

Static Analysis

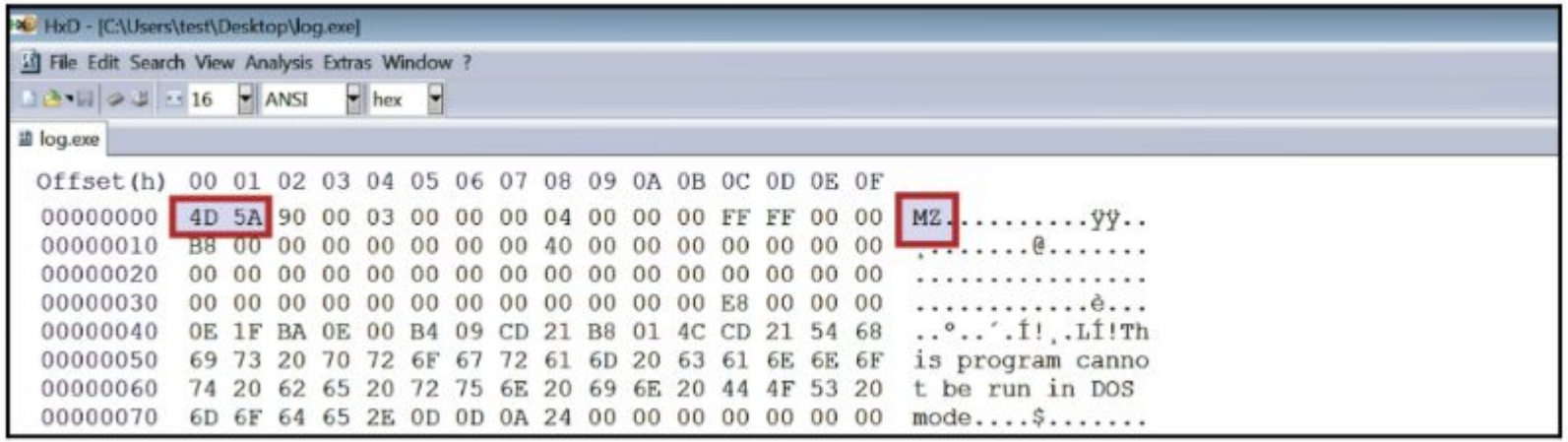
1. Determining File Type:

- ✔ Most Windows-based malware are executable files ending with extensions such as .exe, .dll, .sys, and so on.
- ✔ Instead of relying on file extension, File signature can be used to determine the file type.
- ✔ A file signature is a unique sequence of bytes that is written to the file's header. Different files have different signatures, which can be used to identify the type of file.
- ✔ The Windows executable files, also called PE files (such as the files ending with .exe, .dll, .com, .drv, .sys, and so on), have a file signature of MZ or hexadecimal characters 4D 5A in the first 9-two bytes of the file.

1. Determining File Type:

1.1 Identifying the file type using manual method:

The manual method of determining the file type is to look for the file signature by opening it in a hex editor. A hex editor is a tool that allows an examiner to inspect each byte of the file;most hex editors provide many functionalities that help in the analysis of a file. The following screenshot shows the file signature of MZ in the first two bytes when an executable file is opened with the HxD hex editor (https://mh-nexus.de/en/hxd/):



1.1 Identifying the file type using manual method:

On Linux systems, to look for the file signature, the `xxd` command can be used, which generates a hex dump of the file as shown here:

```
$ xxd -g 1 log.exe | more
0000000: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00  MZ..............
0000010: b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  ........@.......
0000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0000030: 00 00 00 00 00 00 00 00 00 00 00 00 e8 00 00 00  ................
```

1. **file Suspicious_file**
   suspicious.jpg: PE32 executable(GUI) Intel 80386

2. **hexdump –C suspicious_file|head**

3. **binwalk suspicious_file(When malware hides payloads)**

1.2 Identifying the file type using Tools:

On Linux systems, this can be achieved using the *file* utility. In the following example, the file command was run on two different files. From the output, it can be seen that even though the first file does not have any extension, it is detected as a 32-bit executable file (PE32) and the second file is a 64-bit (PE32+) executable:

```
$ file mini
mini: PE32 executable (GUI) Intel 80386, for MS Windows

$ file notepad.exe
notepad.exe: PE32+ executable (GUI) x86-64, for MS Windows
```

On Windows, *CFF Explorer*, part of *Explorer Suite* (http://www.ntcore.com/exsuite.php), can be used to determine the file type; it is not just limited to determining file type. It is also a great tool for inspecting executable files (both 32-bit and 64-bit) and allows you to examine the PE internal structure, modify fields, and extract resources.

1.2 Identifying the file type using Tools:

CFF explorer is a windows-based PE(Portable Executble) analysis tool used in static malware analysis to inspect the internal structure of EXE and Dll files without executing them.

Using CFF explorer is used in Malware Analysis

- ✔ To verify the file signatures
- ✔ Analyze PE haeders
- ✔ View sections(.text, .data, .rsrc)
- ✔ Check imports and Exports

1.3 Identifying the file type using Python:

In Python, the python-magic module can be used to determine the file type. On Windows, to install the python-magic module, you can follow the procedure mentioned at https://github.com/ahupp/python-magic.

Once the `python-magic` is installed, the following commands can be used in the script to determine the file type:

```
$ python
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
>>> import magic
>>> m = magic.open(magic.MAGIC_NONE)
>>> m.load()
>>> ftype = m.file(r'log.exe')
>>> print ftype
PE32 executable (GUI) Intel 80386, for MS Windows
```

Static Analysis

Once the `python-magic` is installed, the following commands can be used in the script to determine the file type:

```
$ python
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
>>> import magic
>>> m = magic.open(magic.MAGIC_NONE)
>>> m.load()
>>> ftype = m.file(r'log.exe')
>>> print ftype
PE32 executable (GUI) Intel 80386, for MS Windows
```

1.3 Identifying the file type using Python:

In Python, the python-magic module can be used to determine the file type. On Windows, to install the python-magic module, you can follow the procedure mentioned at https://github.com/ahupp/python-magic.

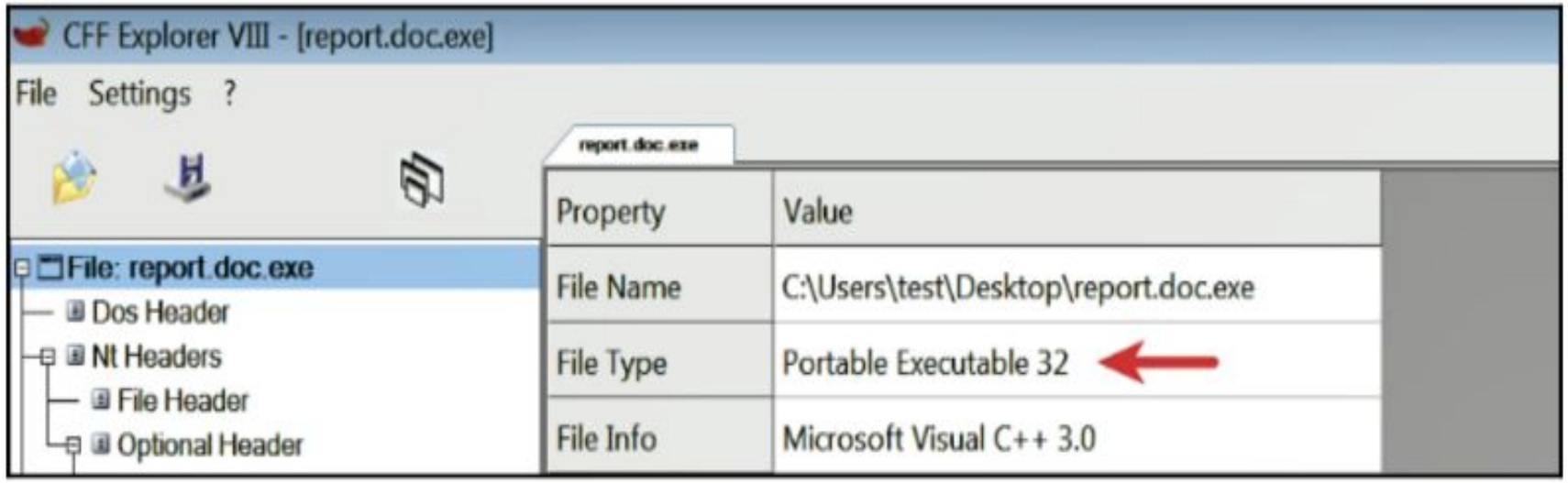
To demonstrate the use of detecting file type, let's take an example of a file that was made to look like a *Word document* by changing the extension from `.exe` to `.doc.exe`. In this case, attackers took advantage of the fact that, by default, *"Hide extension for known file types"* is enabled in the *"Windows folder view options"*; this option prevents the file extension from being displayed to the user. The following screenshot shows the appearance of the file with *"Hide extension for known file types"* enabled:

Static Analysis

To demonstrate the use of detecting file type, let's take an example of a file that was made to look like a *Word document* by changing the extension from `.exe` to `.doc.exe`. In this case, attackers took advantage of the fact that, by default, *"Hide extension for known file types"* is enabled in the *"Windows folder view options"*; this option prevents the file extension from being displayed to the user. The following screenshot shows the appearance of the file with *"Hide extension for known file types"* enabled:

1.3 Identifying the file type using Python:

Opening the file in the *CFF Explorer* reveals that it is a 32-bit executable file and not a word document, as shown here:



Static Analysis

2. Fingerprinting the Malware:

Fingerprinting involves generating the cryptographic hash values for the suspect binary based on its file content. The cryptographic hashing algorithms such as MD5, SHA1 or SHA256 are considered the de facto standard for generating file hashes for the malware specimens.

The following list outlines the use of cryptographic hashes:

- ✔ Identifying a malware specimen based on filename is ineffective because the same malware sample can use different filenames, but the cryptographic hash that is calculated based on the file content will remain the same. Hence, a cryptographic hash for your suspect file serves as a unique identifier throughout the course of analysis.

Static Analysis

2. Fingerprinting the Malware:

- ✔ During dynamic analysis, when malware is executed, it can copy itself to a different location or drop another piece of malware. Having the cryptographic hash of the sample can help in identifying whether the newly dropped/copied sample is the same as the original sample or a different one. This information can assist you in deciding whether the analysis needs to be performed on a single sample or multiple samples.

- ✔ File hash is frequently used as an indicator to share with other security researchers to help them identify the sample. File hash can be used to determine whether the sample has been previously detected by searching online or searching the database of multi Anti-virus scanning service like VirusTotal.

2.1. Generating Cryptographic Hash using tools:

On a Linux system, file hashes can be generated using the `md5sum`, `sha256sum`, and `sha1sum` utilities:

```
$ md5sum log.exe
6e4e030fbd2ee786e1b6b758d5897316  log.exe

$ sha256sum log.exe
01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f  log.exe

$ sha1sum log.exe
625644bacf83a889038e4a283d29204edc0e9b65  log.exe
```

2.1. Generating Cryptographic Hash using tools:

For Windows, various tools for generating file hashes can be found online. *HashMyFiles* (`http://www.nirsoft.net/utils/hash_my_files.html`) is one such tool that generates hash values for single or multiple files, and it also highlights identical hashes with same colors. In the following screenshot, it can be seen that `log.exe` and `bunny.exe` are the same samples based on their hash values:

HashMyFiles

File Edit View Options Help

Filename	MD5	SHA1	SHA-256
log.exe	6e4e030fbd2ee786e1b6b758d5897316	625644bacf83a889038e4a283d29204edc0e9b65	01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f
order.exe	1de7834ba959e734ad701dc18ef0edfc	a8aa7c022cb3cfd2168665cbdaadccd5a1bf0dea	66b08590b515498a974106c69c18be7695e22c4cbec659024c53c6ca90991d8f
SQLite.exe	f8daa49c489f606c87d39a88ab76a1ba	5a12d17152a90eb03c24614d68c7355d36606960	e344ae25471c31f0c3533b69561314e56a12b9c96cf632f17d21126ba5c5521b
bunny.exe	6e4e030fbd2ee786e1b6b758d5897316	625644bacf83a889038e4a283d29204edc0e9b65	01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f

2.1. Generating Cryptographic Hash using tools:

HashMyFiles

File Edit View Options Help

Filename	MD5	SHA1	SHA-256
icg.exe	6e4e030fbd2ee786e1b6b758d5897316	625644bacf83a889038e4a283d29204edc0e9b65	01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f
order.exe	1de7834ba959e734ad701dc18ef0edfc	a8aa7c022cb3cfd2168665cbdaadccd5a1bf0dea	66b08590b515498a974106c69c18be7695e22c4cbec659024c53c6ca90991d8f
SQLite.exe	f8daa49c489f606c87d39a88ab76a1ba	5a12d17152a90eb03c24614d68c7355d36606960	e344ae25471c31f0c3533b69561314e56a12b9c96cf632f17d21126ba5c5521b
bunny.exe	6e4e030fbd2ee786e1b6b758d5897316	625644bacf83a889038e4a283d29204edc0e9b65	01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f

2.2 Determining Cryptographic Hash in Python

In Python, it is possible to generate file hashes using the `hashlib` module, as shown here:

```
$ python
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
>>> import hashlib
>>> content = open(r"log.exe","rb").read()
>>> print hashlib.md5(content).hexdigest()
6e4e030fbd2ee786e1b6b758d5897316
>>> print hashlib.sha256(content).hexdigest()
01636faaae739655bf88b39d21834b7dac923386d2b52efb4142cb278061f97f
>>> print hashlib.sha1(content).hexdigest()
625644bacf83a889038e4a283d29204edc0e9b65
```

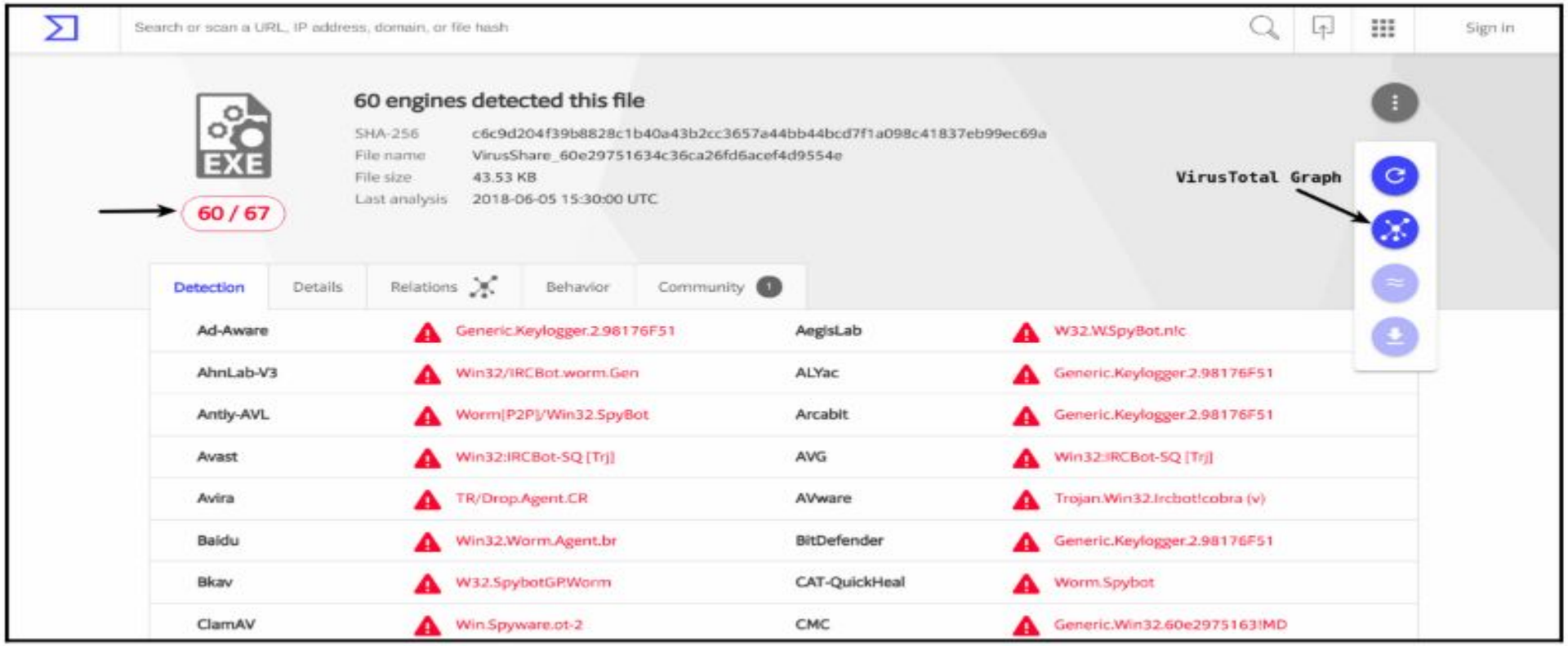
3. Multiple Anti Virus Scanning

- ✔ Scanning the suspect binary with multiple anti-virus scanners helps in determining whether malicious code signatures exist for the suspect file. The signature name for a particular file can provide additional information about the file and its capabilities.

- ✔ By visiting the respective antivirus vendor websites or searching for the signature in search engines, you can yield further details about the suspect file.

3.1. Scanning the suspect binary using VirusTotal

VirusTotal (http://www.virustotal.com) is a popular web-based malware scanning service. It allows you to upload a file, which is then scanned with various anti-virus scanners, and the scan results are presented in real time on the web page.



3.2. Querying Hash values using VirusTotal public API

- VirusTotal is an online service that checks files using many antivirus engines. Instead of uploading a suspicious file, we can query VirusTotal using the file's hash value (MD5, SHA1, or SHA256).
- VirusTotal provides a Public API that allows us to automate this process using scripts (for example, Python). By giving a hash value as input, the script checks whether the file already exists in VirusTotal's database and returns detection results from multiple antivirus vendors.

This method is useful because:

- It avoids uploading sensitive or confidential files.
- It helps analysts quickly know if a file is malicious.
- It shows how many antivirus engines detect the file and what malware name they assign.

However, antivirus results are not 100% reliable, and public scans may expose information to attackers or third parties.

3.2. Querying Hash values using VirusTotal public API

```
import urllib
import urllib2
import json
import sys

hash_value = sys.argv[1]
vt_url = "https://www.virustotal.com/vtapi/v2/file/report"
api_key = "<update your api key here>"
parameters = {'apikey': api_key, 'resource': hash_value}
encoded_parameters = urllib.urlencode(parameters)
request = urllib2.Request(vt_url, encoded_parameters)
response = urllib2.urlopen(request)
json_response = json.loads(response.read())
if json_response['response_code']:
    detections = json_response['positives']
    total = json_response['total']
    scan_results = json_response['scans']
    print "Detections: %s/%s" % (detections, total)

    print "VirusTotal Results:"
    for av_name, av_data in scan_results.items():
        print "\t%s ==> %s" % (av_name, av_data['result'])
else:
    print "No AV Detections For: %s" % hash_value
```

3.2. Querying Hash values using VirusTotal public API

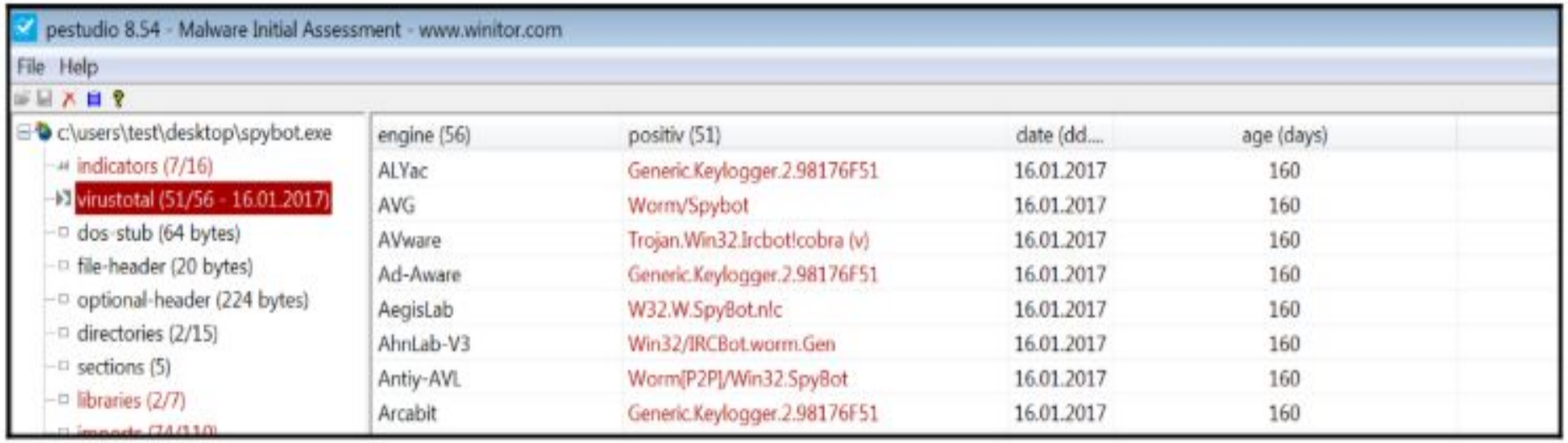
Running the preceding script by giving it an MD5 hash of a binary shows the antivirus detections and the signature names for the binary.

```
$ md5sum 5340.exe
5340fcfb3d2fa263c280e9659d13ba93 5340.exe

$ python vt_hash_query.py 5340fcfb3d2fa263c280e9659d13ba93
Detections: 44/56
VirusTotal Results:
  Bkav ==> None
  MicroWorld-eScan ==> Trojan.Generic.11318045
  nProtect ==> Trojan/W32.Agent.105472.SJ
  CMC ==> None
  CAT-QuickHeal ==> Trojan.Agen.r4
  ALYac ==> Trojan.Generic.11318045
  Malwarebytes ==> None
  Zillya ==> None
  SUPERAntiSpyware ==> None
  TheHacker ==> None
  K7GW ==> Trojan ( 001d37dc1 )
  K7AntiVirus ==> Trojan ( 001d37dc1 )
  NANO-Antivirus ==> Trojan.Win32.Agent.cxbxiy
  F-Prot ==> W32/Etumbot.K
  Symantec ==> Trojan.Zbot
  [.........Removed..............]
```

3.2. Querying Hash values using VirusTotal public API

The other alternative is to use PE analysis tools such as *pestudio* (`https://www.winitor.com/`) or *PPEE* (`https://www.mzrst.com/`). Upon loading the binary, the hash value of the binary is automatically queried from the VirusTotal database and the results are displayed, as shown in the following screenshot:



3.2. Querying Hash values using VirusTotal public API

There are a few factors/risks to consider when scanning a binary with Anti-Virus scanners or when submitting a binary to online anti-virus scanning services:

Antivirus detection is not fully reliable
If a binary file is not detected by antivirus scanners, it does not mean the file is safe. Antivirus tools mainly use signatures and heuristics, and malware authors can modify or obfuscate their code to avoid detection.

Uploading files can expose sensitive data
When a binary is uploaded to a public antivirus scanning service, it may be shared with third parties or vendors. If the file contains confidential, personal, or organizational data, it can lead to information leakage.

Hash-based searching is a safer alternative
Most online antivirus services allow searching using cryptographic hash values such as MD5, SHA1, or SHA256. This method checks existing scan results without uploading the actual binary file.

Scan results are stored and publicly accessible
Once a file is scanned, the results are saved in the service's database and can be queried later. Attackers may check these results to see if their malware was detected and then modify it to bypass future detection.

4 Extracting Strings

Strings provide indicators inside a binary: Strings are readable ASCII or Unicode characters embedded in a file. Extracting these strings can reveal important information such as filenames created by the program, URLs, domain names, IP addresses, registry keys, or attack-related commands used by the malware.

Although strings alone do not give a complete understanding of the malware's purpose, they provide useful hints about what the suspect binary is capable of doing and how it may interact with the system or network.

4.1 String Extraction using tools

To extract strings from a suspect binary, you can use the strings utility on Linux systems. The *strings* command, by default, extracts the ASCII strings that are at least four characters long. With the -a option it is possible to extract strings from the entire file. The following ASCII strings extracted from the malicious binary show reference to an *IP address*. This indicates that when this malware is executed, it probably establishes a connection with that IP address:

```
$ strings -a log.exe
!This program cannot be run in DOS mode.
Rich
.text
`.rdata
@.data
L$"%
h4z@
128.91.34.188
%04d-%02d-%02d %02d:%02d:%02d %s
```

4.1 String Extraction using tools

In the following example, the *ASCII* strings extracted from a malware called *Spybot* give an indication of its *DOS* and *Key logging* capabilities:

```
$ strings -a spybot.exe
!This program cannot be run in DOS mode.
.text
`.bss
.data
.idata
.rsrc
]_^[
keylog.txt
%s (Changed window
Keylogger Started
HH:mm:ss]
[dd:MMM:yyyy,
SynFlooding: %s port: %i delay: %i times:%i.
bla bla blaaaasdasd
Portscanner startip: %s port: %i delay: %ssec.
Portscanner startip: %s port: %i delay: %ssec. logging to: %s
kuang
sub7
%i.%i.%i.0
scan
redirect %s:%i > %s:%i)
Keylogger logging to %s
Keylogger active output to: DCC chat
```

4.1 String Extraction using tools

```
Keylogger active output to: %s
error already logging keys to %s use "stopkeylogger" to stop
startkeylogger
passwords
```

Malware specimens also use *Unicode* (2 bytes per character) strings. To get useful information from the binary, sometimes you need to extract both *ASCII* and *Unicode* strings. To extract Unicode strings using the *strings* command, use the `-el` option.

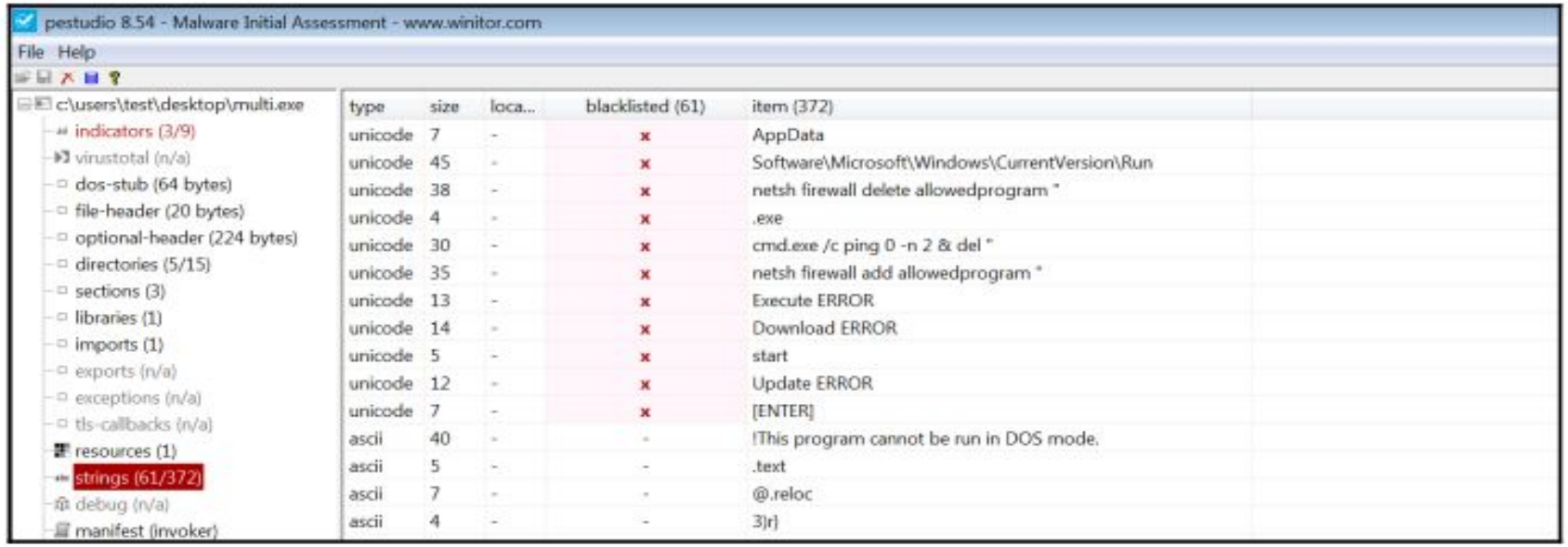
In the following example, the malware sample did not reveal unusual *ASCII* strings, but extracting the *Unicode* strings showed references to the *domain name*, and the *Run registry key* (which is frequently used by malware to survive the reboot); and it also highlights a malware's possible capability to add a program to the firewall whitelist:

```
$ strings -a -el multi.exe
AppData
44859ba2c98feb83b5aab46a9af5fefc
haixxdrekt.dyndns.hu
True
Software\Microsoft\Windows\CurrentVersion\Run
Software\
.exe
SEE_MASK_NOZONECHECKS
netsh firewall add allowedprogram "
```

4.1 String Extraction using tools

- ✔ **pestudio** as a string analysis tool: On Windows, pestudio is a useful PE analysis tool that displays both ASCII and Unicode strings from a suspect executable, helping in the initial assessment of malware.

- ✔ pestudio highlights suspicious or notable strings in a blacklisted column, making it easier for analysts to quickly identify and focus on interesting or potentially malicious strings within the binary.



4.2 Decoding Obfuscated Strings using FLOSS

- ✔ Malware authors often use simple string obfuscation techniques to hide important strings, so these strings do not appear in normal string extraction tools.
- ✔ FireEye Labs Obfuscated String Solver (FLOSS) automatically detects and extracts hidden as well as normal ASCII and Unicode strings, helping analysts reveal information that malware authors try to conceal.

In the following example, running a *FLOSS* standalone *binary* on a malware specimen not only extracted the human-readable strings but also decoded the obfuscated strings and extracted *stack strings* missed by the strings utility and other string extraction tools. The following output shows reference to an *executable*, *Excel file*, and *Run registry key:*

```
$ chmod +x floss
$ ./floss 5340.exe
FLOSS static ASCII strings
!This program cannot be run in DOS mode.
Rich
.text
`.rdata
```

4.2 Decoding Obfuscated Strings using FLOSS

```
@.data
[..removed..]

FLOSS decoded 15 strings
kb71271.log
R6002
- floating point not loaded
\Microsoft
winlogdate.exe
~tasyd3.xls
[....REMOVED....]

FLOSS extracted 13 stack strings
BINARY
ka4a8213.log
afjlfjsskjfslkfjsdlkf
'Clt
~tasyd3.xls
"%s"="%s"
regedit /s %s
[HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run]
[.....REMOVED......]
```

5. Determining File obfuscation

- Malware authors obfuscate or armor their binaries to hide the internal working of the malware from analysts and security researchers, making detection and analysis difficult and resulting in very few or unclear extracted strings.

- Tools like packers and cryptors are commonly used to obfuscate malware, helping it evade antivirus detection and prevent effective string extraction and reverse engineering.

5. Determining File obfuscation

- Malware authors obfuscate or armor their binaries to hide the internal working of the malware from analysts and security researchers, making detection and analysis difficult and resulting in very few or unclear extracted strings.

- Tools like packers and cryptors are commonly used to obfuscate malware, helping it evade antivirus detection and prevent effective string extraction and reverse engineering.

5.1. packers and Cryptors

A packer is a tool that compresses, encrypts, or obfuscates an executable file (.exe) and adds a small loader (stub).

When the program runs:

- ✔ The stub executes first
- ✔ It unpacks/decrypts the real code into memory
- ✔ Then it transfers execution to the original program

5.1. packers and Cryptors

Why packers are used?/

Legitimate reasons

- ✔ Reduce file size
- ✔ Protect intellectual property
- ✔ Prevent reverse engineering

Malicious reasons

- ✔ Hide malware code
- ✔ Evade antivirus signatures
- ✔ Defeat static analysis tools
- ✔ Delay sandbox detection

5.1. packers and Cryptors

A Packer is a program that takes the executable as input, and it uses compression to obfuscate the executable's content. This obfuscated content is then stored within the structure of a new executable file; the result is a new executable file (packed program) with obfuscated content on the disk.

Upon execution of the packed program, it executes a decompression routine, which extracts the original binary in memory during runtime and triggers the execution.

5.1. packers and Cryptors

A Cryptor is similar to a Packer, but instead of using compression, it uses encryption to obfuscate the executable's content, and the encrypted content is stored in the new executable file.

Upon execution of the encrypted program, it runs a decryption routine to extract the original binary in the memory and then triggers the execution.

5.1. packers and Cryptors

A cryptor is a tool that encrypts the malicious code inside an executable and decrypts it only at runtime, usually in memory, to avoid detection.

In simple terms:

Packers compress → Cryptors encrypt

Cryptors are mainly designed to evade antivirus detection, not to reduce file size.

5.1. packers and Cryptors

How a cryptor works??

- ✔ Original malware is encrypted
- ✔ A small decryption stub is added
- ✔ When executed:

  ** Stub runs first

  ** Malware is decrypted in memory

  **Control is transferred to decrypted code

- ✔ Disk file remains encrypted at rest

5.1. packers and Cryptors

Why a cryptor is dangerous??

- ✔ Hide real malware signatures
- ✔ Change encryption keys on every build (FUD – Fully Undetectable)
- ✔ Make each sample look unique
- ✔ Defeat signature-based antivirus

5.1. packers and Cryptors

Difference between packers and Cryptors

Feature	Packer	Cryptor
Purpose	Compression / Obfuscation	Antivirus evasion
Technique	Compress code	Encrypt code
File size	Often smaller	Often larger
Detection	Easier	Harder
Malware usage	Common	Very common
Examples	Upx(Ultimate Packer for Executables), MPRESS,PECompact	FUD(Fully undetectable) Cryptors, Custom Cryptors, Polymorphic

5.1. packers and Cryptors

To demonstrate the concept of file obfuscation, let's take an example of a malware sample called *Spybot* (not packed); extracting strings from *Spybot* show, references to suspicious executable names and IP addresses, as shown here:

```
$ strings -a spybot.exe
[....removed....]
EDU_Hack.exe
Sitebot.exe
Winamp_Installer.exe
PlanetSide.exe
DreamweaverMX_Crack.exe
FlashFXP_Crack.exe
Postal_2_Crack.exe
Red_Faction_2_No-CD_Crack.exe
Renegade_No-CD_Crack.exe
Generals_No-CD_Crack.exe
Norton_Anti-Virus_2002_Crack.exe
Porn.exe
AVP_Crack.exe
zoneallarm_pro_crack.exe
[...REMOVED...]
209.126.201.22
209.126.201.20
```

5.1. packers and Cryptors

The Spybot sample was then run through a popular packer *UPX* (`https://upx.github.io/`), which resulted in a new packed executable file (`spybot_packed.exe`). The following command output shows the size discrepancy between the original and the packed binary. UPX uses compression, because of which the size of the packed binary is lower than the original binary:

```
$ upx -o spybot_packed.exe spybot.exe
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2013
UPX 3.91 Markus Oberhumer, Laszlo Molnar & John Reiser Sep 30th 2013
File size Ratio Format Name
-------------------- ------ ----------- -----------
44576 -> 21536 48.31% win32/pe spybot_packed.exe
Packed 1 file.

$ ls -al
total 76
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 9 09:04 .
drwxr-xr-x 6 ubuntu ubuntu 4096 Jul 9 09:04 ..
-rw-r--r-- 1 ubuntu ubuntu 44576 Oct 22 2014 spybot.exe
-rw-r--r-- 1 ubuntu ubuntu 21536 Oct 22 2014 spybot_packed.exe
```

5.1. packers and Cryptors

Running the strings command on the packed binary shows obscured strings and does not reveal much valuable information; this is one of the reasons why attackers obfuscate their files:

```
$ strings -a spybot_packed.exe
!This program cannot be run in DOS mode.
UPX0
UPX1
.rsrc
3.91
UPX!
t ;t
/t:VU
]^M
9-lh
:A$m
hAgo .
C@@f.
Q*vPCi
%_I;9
PVh29A
[...REMOVED...]
```

5.2 Detecting File Obfuscation using Exeinfo PE

What is file obfuscation / packing?

- ✔ Obfuscation means intentionally making a program hard to read or analyze.
- ✔ In Windows executables (.exe), this is often done using packers (UPX, Themida, MPRESS, etc.).
- ✔ Packers compress or encrypt the real code and add a small stub that unpacks it at runtime.

Why suspicious?

- ✔ Normal software usually doesn't need heavy obfuscation.
- ✔ Malware often uses packers to hide its real behavior from antivirus and analysts.
- ✔ So, when an .exe is packed → red flag in malware analysis.

5.2 Detecting File Obfuscation using Exeinfo PE

What does Exeinfo PE do??

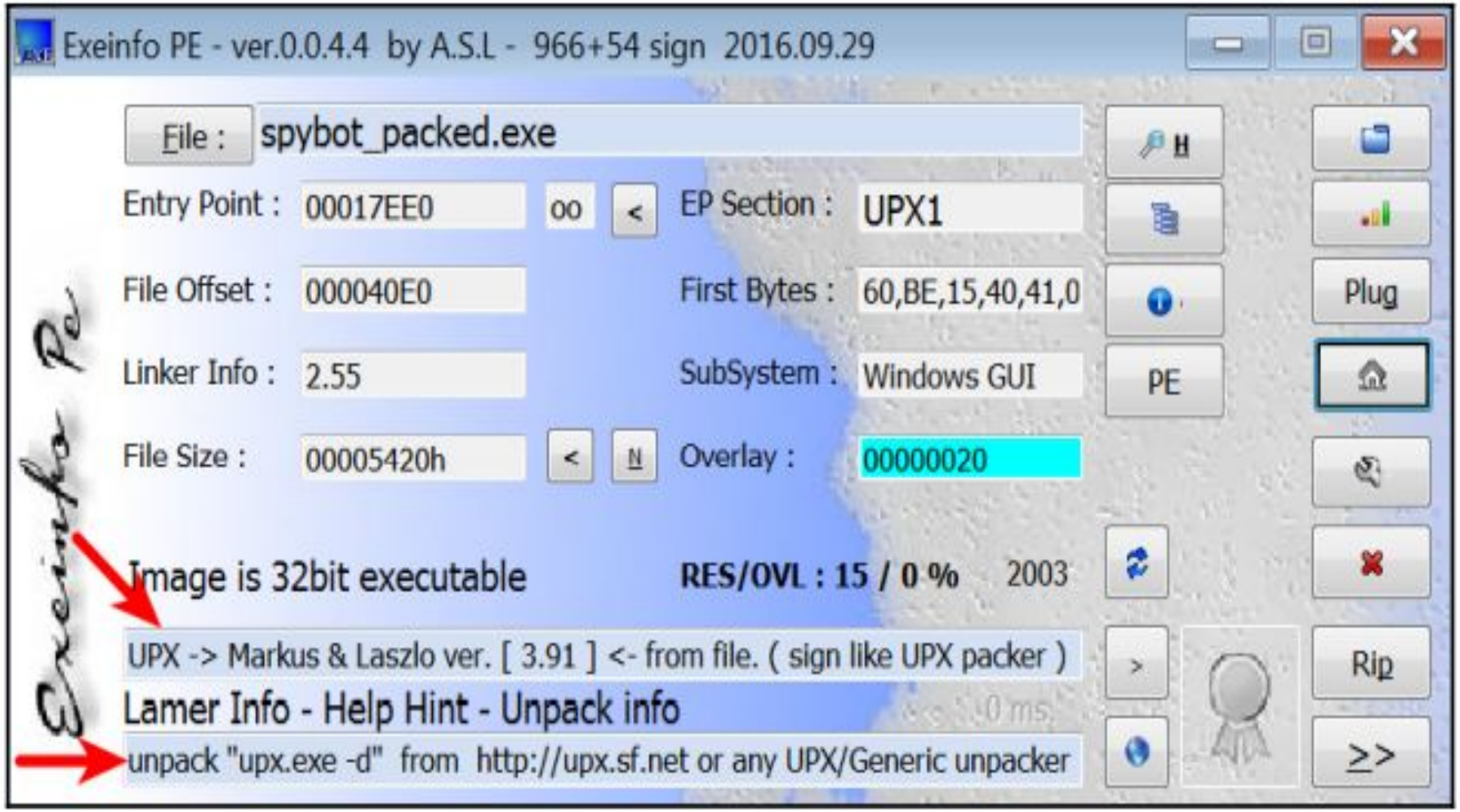
Exeinfo PE is a Windows-only static analysis tool that:

- ✔ Detects if a file is packed or obfuscated
- ✔ Identifies:
  - ** Compiler (GCC, MSVC, etc.)
  - ** Packer (UPX, ASPack, etc.)
  - ** Cryptor / Protector
- ✔ Uses 4500+ signatures to match known packers

5.2 Detecting File Obfuscation using Exeinfo PE

- Most legitimate executables do not use obfuscation, but some may pack their files to protect intellectual property. However, when a sample is found to be packed, especially in malware analysis, it significantly increases the suspicion that the file could be malicious.

- Exeinfo PE is a free Windows-based tool with a user-friendly graphical interface. It uses more than 4,500 known signatures to identify the compilers, packers, or cryptors used to build an executable, making it effective for detecting packed malware.

- In addition to identifying the packer (such as UPX), Exeinfo PE provides useful information and references on how to unpack or decompress the obfuscated file. This feature helps analysts reverse the packing process and simplifies further malware analysis.

5.2 Detecting File Obfuscation using Exeinfo PE